

Retrieval-Augmented Generation Fundamentals

pig7selene

September 5, 2026

Abstract

Retrieval-Augmented Generation (RAG) changes the information available to a language model at inference time. An offline pipeline converts an external corpus into retrievable units and an index; an online pipeline ranks units for a query, constructs a bounded context, and asks the language model to generate from that context. This chapter establishes the resulting system vocabulary and separates retrieval success, context-construction quality, and generation quality. It also explains freshness, provenance, context limits, and the boundary between RAG and fine-tuning, providing a foundation for later chapters on retrievers, reranking, and RAG evaluation.

1 Introduction

A pretrained language model encodes regularities from its training distribution in parameters θ . This **parametric knowledge** is useful precisely because the model can apply it through ordinary next-token prediction, but it is not an externally addressable database. A model cannot normally expose the source record for an individual learned association, replace one fact without changing parameters, or guarantee that its training snapshot includes a newly published document. Fine-tuning can change the model, but it is an optimization procedure rather than a per-query lookup mechanism.

Retrieval-Augmented Generation (RAG) adds an external knowledge path. Before an answer is generated, a system retrieves text or another structured knowledge unit from a maintained corpus and places selected evidence in the model's input context. The foundational RAG formulation combines a parametric generator with a non-parametric retrievable memory, motivated in part by updateability and provenance on knowledge-intensive tasks [1]. In contemporary systems, the retriever and generator need not be trained jointly; the architectural idea is broader than one particular model family.

This chapter gives the system model that later RAG chapters refine. It distinguishes documents, passages, chunks, retrieval scores, ranking, context construction, and grounded generation. It does not derive BM25, embedding similarity, approximate nearest-neighbor search, reranking, query rewriting, or RAG evaluation protocols. Those are later design choices inside the interfaces defined here. Chapter 1 supplies the tokenizer contract that turns retrieved text into context tokens, Chapter 19 supplies the autoregressive inference model, and Chapter 25 supplies the end-to-end systems perspective needed once retrieval becomes part of a serving path.

2 RAG as an External Knowledge System

Let

$$D = \{d_1, \dots, d_N\} \tag{1}$$

be the retrieval corpus. Each d_i is a retrieval unit, not necessarily one original file: it may be a complete document, a passage, a chunk cut from a larger document, or a structured record rendered as text. A source identifier and metadata connect d_i back to its provenance. Let q be a user query after the system has applied its declared normalization and tokenization conventions.

The RAG architecture has two time scales. **Offline indexing** prepares a corpus before user requests arrive. It parses source material, constructs retrieval units, produces the representations required by a

chosen retriever, and stores an index plus metadata. **Online retrieval** receives q , scores or filters units from the prepared corpus, selects candidates, constructs a context, and invokes an LLM. Separating the two paths matters: a change to a source document usually requires reprocessing and index publication, but does not require retraining the base model.

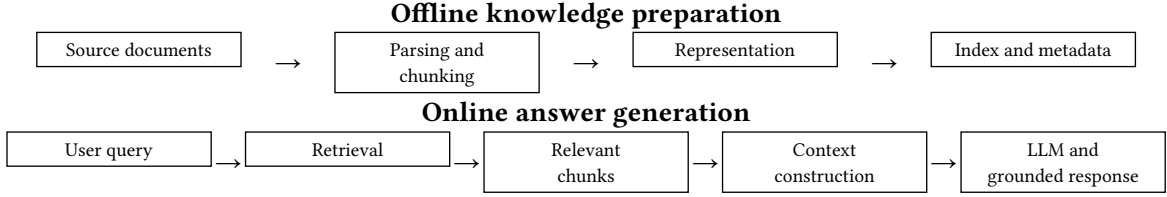


Figure 1: RAG separates corpus preparation from query-time augmentation. The index makes units retrievable; the context builder turns selected units into the bounded model input that the generator actually sees.

The generator remains an autoregressive language model. If $C(q)$ is the context constructed for a query, then a conceptual answer distribution is

$$p_{\theta}(y \mid q, C(q)). \quad (2)$$

Equation 2 does not claim that every generated token is supported by $C(q)$. It makes the intended information path explicit: retrieval changes a conditioning input at inference time, while ordinary generation still depends on the model parameters, the prompt template, and the decoding policy. A system is grounded only to the degree that it retrieves appropriate evidence, presents it intelligibly, and causes the generator to use it.

3 Corpus Units and Offline Indexing

The original source corpus and the retrieval corpus should not be treated as identical by default. A source document may contain headings, tables, boilerplate, document-level metadata, and many independently useful claims. Parsing extracts a normalized representation. Chunking then produces retrieval units whose size and boundaries make retrieval and later context use practical. A **passage** often denotes a coherent span; a **chunk** emphasizes a unit selected by an implementation rule. Either can be an element d_i of D in Equation 1.

Each unit should retain at least a stable identifier, its source-document identity, a location or span when meaningful, the text used for representation, and version or freshness metadata. These fields are not secondary bookkeeping. They permit a retrieved chunk to be deduplicated, ordered, filtered, attributed, and invalidated when the underlying source changes. Without them, a system can retrieve text but cannot reliably say which document it came from or whether an answer cites the right revision.

An indexing pipeline produces a searchable representation for every retained unit. The representation may be lexical, sparse, dense, or a combination. Lexical retrieval uses observable terms and their statistics; sparse learned representations retain a high-dimensional feature view; dense retrieval represents queries and units through compact learned vectors; hybrid retrieval combines signals. Information-retrieval literature treats ranking as a distinct problem of matching an information need to corpus items, rather than a property of the generator alone [2]. Dense Passage Retrieval is an example of learning query and passage representations for efficient top- k selection, while sparse systems such as BM25 remain important baselines and components [3], [4].

The choice of representation affects what becomes easy to retrieve, but all variants require a corpus contract. The index must identify the corpus version, parsing rules, retrieval-unit boundaries, representation-model revision when applicable, and the fields that were searchable. An index built before a source deletion, a parser repair, or a policy change may be syntactically valid while operationally stale.

4 Online Retrieval and Ranking

At query time, a retriever assigns a relevance score

$$s(q, d_i) \quad (3)$$

to an eligible retrieval unit. The score need not be a calibrated probability, and scores from different retrievers need not share a common numerical scale. Its operational role is ordinal: rank candidates for this query. Let $R_{k(q)}$ be the ordered top- k result set,

$$R_{k(q)} = \text{TopK}_{d_i \in D} s(q, d_i) = (d_{\pi_1}, \dots, d_{\pi_k}), \quad (4)$$

where $\pi_1, \dots, \pi_k$ order the selected units from higher to lower score under the declared tie rule. Eligibility can include access control, language, recency, document type, tenant, or metadata filters. These filters may prevent an unsuitable unit from reaching the generator even when its unfiltered score is high; they are therefore part of retrieval semantics, not an afterthought.

Retrieval does not imply dense vector search. A lexical system can rank documents by term evidence; a sparse system can score learned sparse features; a dense system can compare learned query and document representations; and a hybrid system can combine or stage those signals. This chapter deliberately treats s as an abstract interface because later chapters must examine how each family represents and searches the corpus. Replacing one scoring method with another can change both the candidate distribution and the kinds of failure a generator encounters.

First-stage retrieval is often asked to preserve plausible evidence among a relatively small candidate set rather than to make a final answer decision. A ranking miss cannot be repaired by a later context builder or generator, because the required evidence was never made available. Conversely, a high-ranked unit can be topically related yet fail to contain the exact fact, scope condition, or source authority needed for a correct response.

5 Context Construction and Grounded Generation

Retrieval returns candidate units; it does not yet define the model input. A **context constructor** selects, deduplicates, orders, labels, and serializes candidates for the LLM. With metadata m_i for unit d_i , a conceptual construction is

$$C(q) = \text{compose}(q, (d_{\pi_1}, m_{\pi_1}), \dots, (d_{\pi_k}, m_{\pi_k})). \quad (5)$$

The constructor may include document titles, URLs or stable source IDs, dates, section labels, and delimiters that make evidence boundaries visible to the model. Its order can influence attention and generation. Duplicate passages waste scarce input capacity and can make one claim appear more strongly supported than it is. Conflicting passages need their source identity and temporal scope preserved; silently concatenating them does not resolve the conflict.

Context capacity imposes a hard systems boundary. Let τ be the tokenizer from Chapter 1, let $T_{\max}$ be the model’s allowed context length, and let N_{out} be a reserved output budget. A request must satisfy a constraint of the form

$$|\tau(C(q))| + N_{\text{out}} \leq T_{\max}. \quad (6)$$

The exact accounting also includes any system instructions, role tokens, and output-format markers. This is why retrieving more documents is not automatically better. Additional material can displace stronger evidence, increase Prefill cost, consume context capacity, or distract the generator. A long context window changes the feasible budget but does not remove the need to identify relevant evidence: an entire corpus may still be too large, and irrelevant context still has a cost.

Boundary	Input and output	Necessary success condition
Retrieval	Query $q \rightarrow$ ranked units $R_{k(q)}$	At least one suitably authoritative and relevant unit survives the candidate boundary.
Context construction	Ranked units $\rightarrow$ serialized context $C(q)$	Useful evidence fits, remains identifiable, and is not obscured by duplicates, conflicts, or irrelevant material.
Generation	Query and context $\rightarrow$ answer y	The model follows the evidence, handles uncertainty, and does not invent unsupported claims or citations.

Table 1: RAG quality is compositional. Success at one boundary is necessary but insufficient for success at the next boundary.

Grounded generation is an intended behavioral property, not a consequence automatically supplied by retrieval. A generator can ignore a relevant chunk, combine claims from conflicting sources incorrectly, answer from parametric memory instead of supplied evidence, or manufacture an unsupported citation. Conversely, an accurate answer can be generated after imperfect retrieval if another selected unit contains enough evidence. The provenance shown to a user must therefore be tied to the actual units used in $C(q)$ and evaluated as a claim about support, not merely as a list of nearby links.

6 Retrieval Quality, Freshness, and Provenance

Retrieval evaluation begins with a relevance relation rather than with answer fluency. For a query q , let $G(q) \subset D$ be the set of units judged relevant under a declared task definition. A passage-level Recall@ k can be written as

$$\text{Recall @}k(q) = \frac{|R_{k(q)} \cap G(q)|}{|G(q)|}, \quad (7)$$

when $G(q)$ is nonempty. Precision@ k is

$$\text{Precision @}k(q) = \frac{|R_{k(q)} \cap G(q)|}{k}. \quad (8)$$

Recall asks how much relevant evidence reaches the candidate set; precision asks what fraction of the returned material is relevant. In tasks with one or a few acceptable evidence passages, a binary Hit@ k indicator, $1[R_{k(q)} \cap G(q) \neq \emptyset]$, is also common. The metrics are useful only with carefully defined relevance judgments: one source can contain an answer string without supporting the required interpretation, and several sources can be relevant but differ in authority or freshness.

Recall is especially important at the first-stage boundary. If $R_{k(q)}$ excludes all useful evidence, downstream ranking or generation cannot recover the omission without another retrieval attempt. Precision also matters because the context budget in Equation 6 is finite. The balance depends on the later pipeline: a broad first stage may favor recall before a later reranker, whereas a direct retrieve-and-generate path may need stronger top- k precision. Ranking quality also depends on position, because a relevant unit at rank one and the same unit at rank k do not have identical chances of surviving a budgeted context constructor.

RAG’s freshness advantage follows from separating the corpus from θ . A newly available document can be parsed, chunked, represented, and published in a new index version; it can then become retrievable without a base-model update. This property is valuable, but it is not a guarantee of current or correct answers. The ingestion pipeline can lag, the index can retain an old version, the query can retrieve obsolete evidence, and the generator can still ignore a current source. A responsible system records the source version and retrieval time alongside any freshness claim.

Provenance is similarly an interface, not a decorative citation feature. A response citation should identify a source document and, when possible, the chunk or span that supplied the asserted evidence. It should not be synthesized from a model’s confidence or from a document that was merely retrieved but excluded from $C(q)$. Traceability enables auditing, correction, and source-specific policy enforcement; it does not establish that the cited text entails the answer.

7 RAG and Fine-Tuning

RAG and fine-tuning modify different parts of a system. RAG primarily changes the information presented at inference time through $C(q)$ in Equation 2. Fine-tuning primarily changes parameters θ or the behavior learned from a training distribution. A fine-tuned model may follow a domain-specific response format more reliably, use a tool convention, or reason in a desired style even with no retrieved text. A RAG system can expose a newly indexed policy or manual without modifying the base model.

Question	RAG	Fine-tuning
What changes?	Runtime context and access to external units	Model parameters or learned behavior
How is a new source made available?	Update the corpus and publish an index version	Collect data and run an adaptation procedure
What can be inspected at answer time?	Retrieved units, context serialization, and provenance records	Checkpoint and training provenance, not a per-answer source lookup
What does it not guarantee?	Correct use of retrieved evidence or correct citations	Fresh factual coverage or a traceable source for each answer

Table 2: RAG and fine-tuning have different control surfaces. They are often complementary: adaptation can improve how a model consumes evidence, while retrieval supplies current and inspectable task information.

The two techniques are therefore complementary rather than substitutes. An organization can fine-tune a model for instruction following or a specialized output schema while retrieving current documents at runtime. It can also improve retrieval with learned representations without changing the generator. The appropriate boundary depends on update frequency, corpus scale, provenance requirements, latency budget, data governance, and the behavior expected when evidence is absent or conflicting.

8 Failure Modes

A **retrieval miss** occurs when the candidate set omits useful evidence. It may arise from incomplete ingestion, an unsuitable retrieval unit, query vocabulary mismatch, an inappropriate filter, or poor ranking. A **context failure** occurs when useful retrieved material is truncated, duplicated, ordered poorly, stripped of its identity, or overwhelmed by conflicting and irrelevant passages. A **generation failure** occurs when the LLM ignores, misreads, overgeneralizes, or contradicts evidence that was available in its context. These boundaries suggest different repairs; asking the generator to be more factual cannot restore a document that the retriever never returned.

Staleness is a cross-cutting failure. An index can contain an obsolete source, a newer source can be absent, or a version change can leave derived representations inconsistent with raw text. Bad chunk boundaries can sever a condition from the statement it qualifies, while duplicate evidence can inflate a weak claim’s apparent support. A context overflow can exclude the best unit even after successful ranking. Citation failure is more specific still: an answer can retrieve correct material yet cite an unrelated source, cite a source that does not support the claim, or fabricate a citation entirely.

The practical lesson is not that every RAG system requires every possible safeguard. It is that observability must preserve the boundaries in Table 1. A production trace should make it possible to determine whether a bad answer began with a corpus omission, a retrieval ranking decision, a context-construction decision, or generation conditioned on an already flawed prompt.

9 Implementation Contracts

The **corpus contract** must version source identities, access policy, parsing rules, chunk boundaries, unit IDs, metadata schema, deduplication policy, representation inputs, index revision, and publication time. A replacement or deletion must specify whether old units are removed, superseded, or retained with an explicit validity interval. The contract should make a source document’s path to one or more d_i units auditable.

The **retrieval contract** must state the query normalization and tokenizer, retriever revision, filters, score semantics, tie rule, candidate count k , and the identifiers and scores returned in $R_{k(q)}$. It should preserve enough trace information to reproduce a result against the same index version. If access control or tenant filters are applied, the system must record them without leaking forbidden corpus content into prompts or logs.

The **context and generation contract** must state the prompt template, source-label format, deduplication rule, ordering policy, context budget, reserved output budget, model revision, decoding policy, and citation-rendering rule. Tests should verify that the serialized context satisfies Equation 6, that each displayed citation maps to a retained source unit, and that a removed or unauthorized unit cannot survive into $C(q)$. Evaluation must separate retrieval coverage, context validity, answer correctness, grounded support, and citation correctness rather than treating a fluent final answer as evidence that every upstream boundary worked.

10 Summary

RAG augments a language model with an external information path. An offline pipeline turns source documents into parsed, versioned retrieval units and an index. At query time, a retriever ranks corpus units, a context constructor turns selected evidence into a bounded model input, and an autoregressive generator produces a response conditioned on that input. The score $s(q, d_i)$ and top- k set $R_{k(q)}$ define a retrieval boundary; the constructed context $C(q)$ defines a separate prompt boundary; the answer distribution in Equation 2 defines the generation boundary.

Those boundaries explain both RAG’s value and its limits. Updating an external corpus can make information available without retraining the base model, while metadata and source identity can make the answer path inspectable. Neither property guarantees a correct, grounded, or correctly cited answer. High retrieval recall cannot ensure sound context construction, and strong generation cannot use evidence that was never retrieved. Later chapters will develop the retrieval methods, context-selection mechanisms, and evaluation procedures needed to make these interfaces reliable.

References

- [1] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems 33*, Curran Associates, Inc., 2020, pp. 9459–9474.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1017/CBO9780511809071.
- [3] V. Karpukhin *et al.*, “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 6769–6781. doi: 10.18653/v1/2020.emnlp-main.550.
- [4] S. E. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/15000000019.